

MODULE 36

AI Agents and Search Engine with LangChain Tools

Lectures Covered

- Lecture 1 — Tools in AI Agents and Built-In Tools
- Lecture 2 — Creating Custom Tools and Toolkit
 - Lecture 3 — Tool Binding for Tool Calling
- Lecture 4 — Tool Calling and Tool Execution
 - Lecture 5 — What is an AI Agent
 - Lecture 6 — Creating an AI Agent
 - Lecture 7 — Working of a ReAct AI Agent

Module 36 AI Agents and Search Engine with LangChain Tools | 7 lectures · 4h 26m

AI Agents use an LLM as a reasoning engine to decide which tools to call, in what order, based on the task. Tools are functions the LLM can invoke. LangChain provides built-in tools, custom tool creation, and full agent frameworks.

Lec 1 — Tools in AI Agents and Built-In Tools

```
from langchain_community.tools import WikipediaQueryRun, DuckDuckGoSearchRun
from langchain_community.utilities import WikipediaAPIWrapper

# Built-in search tools
search = DuckDuckGoSearchRun() # web search, no API key needed
wiki = WikipediaQueryRun(api_wrapper=WikipediaAPIWrapper())

print(search.invoke('latest AI news 2024'))
print(wiki.invoke('Transformer neural network'))

# Other built-in tools
from langchain_community.tools import ArxivQueryRun
from langchain_community.utilities import ArxivAPIWrapper
arxiv = ArxivQueryRun(api_wrapper=ArxivAPIWrapper())
print(arxiv.invoke('attention is all you need'))
```

Lec 2 — Creating Custom Tools and Toolkit

```
from langchain_core.tools import tool
from langchain.tools import Tool, StructuredTool
from pydantic import BaseModel, Field

# Method 1: @tool decorator (simplest)
@tool
def get_word_count(text: str) -> int:
    '''Returns the number of words in a text string.'''
    return len(text.split())

# Method 2: Tool from function
def celsius_to_fahrenheit(celsius: str) -> str:
    return f'{float(celsius) * 9/5 + 32:.1f}F'

temp_tool = Tool(
    name = 'TemperatureConverter',
    func = celsius_to_fahrenheit,
    description = 'Converts Celsius to Fahrenheit. Input: number as string.'
)

# Method 3: StructuredTool with Pydantic schema
class SearchInput(BaseModel):
    query: str = Field(description='Search query')
    max_results: int = Field(default=3, description='Max results to return')

def search_func(query: str, max_results: int = 3) -> str:
    results = DuckDuckGoSearchRun().invoke(query)
    return results[:500 * max_results]

structured_search = StructuredTool.from_function(
    func=search_func, name='WebSearch', args_schema=SearchInput,
    description='Searches the web for information.'
)

# Toolkit: group related tools
my_toolkit = [get_word_count, temp_tool, structured_search]
```

Lec 3 — Tool Binding for Tool Calling

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model='gpt-4o')
tools = [search, wiki, arxiv, get_word_count]
```

```
# Bind tools to the model - model now knows about available tools
llm_with_tools = llm.bind_tools(tools)

# Model returns tool call requests instead of text when needed
response = llm_with_tools.invoke('Search for latest LangChain updates')
print(response.tool_calls) # [{'name': 'duckduckgo_search', 'args': {...}}]
print(response.content)    # empty when tool call requested
```

Lec 4 — Tool Calling and Tool Execution

```
from langchain_core.messages import HumanMessage, ToolMessage

# Manual tool execution loop
messages = [HumanMessage('What is the weather in Paris? Search online.')]
response = llm_with_tools.invoke(messages)
messages.append(response)

# Execute any tool calls the model requested
tool_map = {t.name: t for t in tools}
for tc in response.tool_calls:
    tool_result = tool_map[tc['name']].invoke(tc['args'])
    messages.append(ToolMessage(content=str(tool_result), tool_call_id=tc['id']))

# Send results back - model gives final answer
final = llm_with_tools.invoke(messages)
print(final.content)
```

Lec 5 — What is an AI Agent

Agent = LLM + Tools + Loop
A standard LLM call is one-shot: input -> output.
An Agent is a loop: LLM decides -> call tool -> observe result -> LLM decides again -> ...
The LLM acts as the 'brain' — it reasons about WHICH tool to use and WHEN to stop.
The loop continues until the LLM decides it has enough information to give a final answer.
Key components: LLM (reasoner), Tools (actions), Memory (state), Prompt (instructions).

Component	Role Example
LLM (brain)	Decides next action based on observations GPT-4o, LLaMA 3
Tools	Actions the agent can perform web search, calculator, SQL, API call
Memory	Stores conversation + tool results AgentExecutor message history
Agent Prompt	Instructs the LLM how to reason and use tools ReAct template
Stop condition	LLM outputs final answer when done 'Final Answer: ...'

Lec 6 — Creating an AI Agent

```
from langchain_openai import ChatOpenAI
from langchain.agents import create_react_agent, AgentExecutor
from langchain_community.tools import DuckDuckGoSearchRun, WikipediaQueryRun
from langchain_community.utilities import WikipediaAPIWrapper
from langchain import hub

llm = ChatOpenAI(model='gpt-4o', temperature=0)
tools = [
    DuckDuckGoSearchRun(),
    WikipediaQueryRun(api_wrapper=WikipediaAPIWrapper()),
    arxiv,
    get_word_count,
]

# Pull standard ReAct prompt from LangChain Hub
prompt = hub.pull('hwchase17/react')

# Create ReAct agent
agent = create_react_agent(llm, tools, prompt)
executor = AgentExecutor(
    agent=agent,
    tools=tools,
```

```

    verbose          = True,      # prints Thought/Action/Observation trace
    max_iterations   = 10,        # safety limit
    handle_parsing_errors=True
)

result = executor.invoke({'input': 'Who invented the Transformer and what year?'})
print(result['output'])

```

Lec 7 — Working of a ReAct AI Agent

ReAct Loop — Reason + Act

1. **THOUGHT:** LLM reasons about what to do next
2. **ACTION:** LLM picks a tool and its input
3. **OBSERVATION:** Tool executes, result returned to LLM
4. **THOUGHT:** LLM reasons about the observation
5. ... repeat until LLM has enough info ...
6. **FINAL ANSWER:** LLM outputs the answer and loop stops

```

# Example trace (verbose=True output):

# Input: 'What year was the Transformer paper published and who wrote it?'
#
# Thought: I should search for the Transformer paper.
# Action: duckduckgo_search
# Action Input: 'Transformer neural network paper year authors'
# Observation: 'Attention Is All You Need, Vaswani et al., 2017...'
#
# Thought: I now have the answer.
# Final Answer: The Transformer paper 'Attention Is All You Need'
#               was published in 2017 by Vaswani et al. at Google.

# OpenAI Tools agent (modern alternative to ReAct)
from langchain.agents import create_openai_tools_agent
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder

prompt = ChatPromptTemplate.from_messages([
    ('system', 'You are a helpful assistant with access to tools.'),
    ('human', '{input}'),
    MessagesPlaceholder('agent_scratchpad') # holds tool call history
])

agent = create_openai_tools_agent(llm, tools, prompt)
executor = AgentExecutor(agent=agent, tools=tools, verbose=True)
result = executor.invoke({'input': 'Find the latest LangChain version'})
print(result['output'])

```

ReAct vs OpenAI Tools Agent

ReAct (hwchase17/react): text-based reasoning; works with any LLM; slower.
 OpenAI Tools Agent: uses native function calling; faster; requires GPT-4/3.5.
 Use OpenAI Tools Agent for production — more reliable and faster.
 Use ReAct for non-OpenAI models (Groq, Ollama, HuggingFace).

End of Modules 36 Notes — Generative AI Course